A Verification-Gated, Adversarially-Robust Agentic Flow for Open-Source IC Design

IEEE SSCS PICO Open-Source Chipathon 2026 ， Track D (AI / LLM-assisted Circuits)

Project Proposal, May 2026

Primary contribution: the secure agentic flow.

PRESENT-80 on GF180MCUD.

Table of Contents

Executive Summary

We propose to build, evaluate, and open-source a **hardened agentic pipeline for RTL-to-GDS chip design**, a system whose architecture is explicitly designed to resist adversarial manipulation of the design specification. The pipeline decomposes the RTL-to-GDS flow into five sandboxed stages, coordinated by a top-level orchestrator and implemented by subagents within each stage. Stage transitions are gated by verification oracles (functional simulation, formal equivalence, lint, DRC, LVS), preceded by a static pre-flight audit (Gate 0) of the agent configuration itself, performed by a from-scratch static scanner we build and release as a standalone package alongside the pipeline. The central research contribution is the pipeline's adversarial robustness, not the chip it produces.

To demonstrate that the pipeline works end-to-end and produces real hardware, we use it to design and tape out a **PRESENT-80 block-cipher core on GF180MCUD** , a small, well-specified digital block chosen because its published test vectors and formal security properties make it easy to detect when the pipeline has been manipulated into producing a subtly incorrect or malicious implementation. The chip is the demonstration vehicle, not the contribution.

We evaluate the hardened pipeline against an unhardened baseline and a per-defense-layer ablation series, using a red-team corpus instantiating an attack taxonomy derived from the first author's prior agent-security research (manuscripts in submission) and grounded in the published canon (OWASP LLM Top 10, Greshake et al. on indirect prompt injection, ToolHijacker, MUZZLE, the SoK on agentic AI attack surface). We measure Attack Success Rate, Unsafe Action Rate, Tapeout Survival, a stage-of-detection breakdown across all gates, and a per-defense-layer contribution table.

The pipeline is model-agnostic by construction, no prompt, tool schema, or output parser depends on a specific model. The evaluation runs across a panel of state-of-the-art models spanning open-source and closed-source families, to test whether security results generalize across model families and to rule out single-model RLHF artifacts. A secondary three-subagent Attacker/Defender/Auditor pipeline (run with one SOTA model from the panel) provides a generative adversarial signal not constrained to the frozen corpus. All LLM calls run at temperature 0 with frozen output artifacts, ensuring full corpus replayability of the evaluation.

This proposal makes a clear distinction between two research problems that are related but separate: **secure agentic flows**, the focus of this work , and **secure chip design**, which appears here only as a stretch goal. To our knowledge this is the first integration of agent-security methodology with a reproducible open-source IC design contest, producing both a fabricable tapeout and a paper-quality agentic robustness study.

The urgency is grounded in the published agent-security literature. Prompt injection is ranked #1 on the OWASP Top 10 for LLM Applications. Indirect injection through untrusted inputs (Greshake et al.) and tool-selection manipulation (ToolHijacker) are reproducible attack surfaces against deployed agentic systems. The SoK on agentic AI

attack surface catalogs the long-horizon failure modes that arise specifically from tool-using agents , the regime EDA pipelines operate in. EDA toolchains, which ingest untrusted specifications, datasheets, and reference designs, and which invoke tools with broad system access, are an obvious extension of this attack surface , one that has not yet received systematic security study.

1. Background and Motivation

1.1 The two problems: secure agentic flows vs. secure chip design

This proposal addresses one of two distinct research problems that are frequently conflated in discussions of AI and hardware security. It is worth being precise about the difference upfront, because the contribution, the threat model, the evaluation methodology, and the publication venue all depend on which problem is being solved.

Secure chip design asks: *does the chip that comes out of the design flow have the security properties we intended?* This is the domain of hardware Trojan detection, side-channel resistance, formal property verification of the RTL, and supply-chain integrity of the IP blocks used. The adversary here is an attacker who wants the chip to misbehave , perhaps by exfiltrating data, weakening a cryptographic primitive, or activating a backdoor under a trigger condition. This is a well-studied field with a large literature (TCHES, HOST, IEEE TDSC, DAC security track) and the attack surface is the hardware artifact itself.

Secure agentic flows asks: *can an adversary manipulate the AI-based design pipeline into producing a chip it was not supposed to produce?* The adversary here does not attack the chip directly , they attack the agent responsible for building it. The threat is a malicious specification, a poisoned reference document, or a prompt-injection payload embedded in an input that appears legitimate to a human reviewer. If the agent is successfully manipulated, the output chip may have been subtly degraded, back-doored, or may have bypassed the verification checks that were supposed to catch exactly this. This is fundamentally an agent-security and AI robustness problem that happens to manifest in the EDA domain. The literature on it is sparse to nonexistent.

This proposal focuses entirely on secure agentic flows. The chip (PRESENT-80) exists to give the agent something real to build and to provide a ground truth for evaluating whether the agent was successfully manipulated into producing something incorrect. Secure chip design , specifically, formal verification of the produced chip's security properties , appears as a stretch goal that the hardened pipeline makes possible, but it is not the contribution.

1.2 Contest context

The IEEE SSCS PICO Open-Source Chipathon 2026 is the sixth edition of the contest, sponsored by the OpenROAD Initiative. Track D , *AI / LLM-assisted Circuits* , focuses on agentic design flows, generator-based methodologies, design-space exploration, and

reproducible design approaches, with explicit emphasis on reproducibility, verification artifacts, and benchmark metrics. Teams target the open-source GF180MCUD PDK. The program culminates in a workshop with a publication pathway.

1.3 The research gap

Surveying the relevant literature reveals five clusters of work that, together, leave a clear opening:

1. **Agentic RTL generation frameworks** (MAGE, AIvril2, CoopetitiveV, ChipCraftBrain, ChipAgents) optimize functional pass rates on benchmarks like VerilogEval, RTLLM, CVDP. **None evaluates adversarial robustness of the pipeline itself.**
2. **LLM hardware-security benchmarks** (HardSecBench, VerilogLAVD, SecV) evaluate one-shot generation against CWE-coded vulnerabilities in the output RTL. **They assess the chip's security properties, not the pipeline's resilience to adversarial inputs.**
3. **LLM-Trojan work** (SENTAUR, RTL-Breaker, TrojanWhisper) treats hardware Trojans as offline design artifacts or training-data backdoors. **None frames the scenario where the running agent is hijacked mid-flow into producing a Trojan-bearing tapeout.**
4. **General agent-security work** (Greshake et al. on indirect prompt injection, ToolHijacker, MUZZLE, ASTRIDE, the SoK on agentic AI attack surface, Beurer-Kellner et al. on agent design patterns) has thoroughly studied web agents and coding assistants. **It has not been instantiated for EDA toolchains, which have distinct tool-dispatch surfaces and hardware-artifact outputs.**
5. **Static agent-configuration analysis** is an emerging area , informed in part by the first author's prior taxonomy work in submission , but typically appears as a standalone scanner rather than as an integrated pre-flight gate inside a domain-specific pipeline. **No published work integrates static config auditing with verification-gated tool dispatch in an EDA flow.**

Our contribution sits at the intersection of clusters 1, 4, and 5: the first threat model and red-team evaluation of agentic RTL pipelines under adversarial input, with a defended reference implementation that produces a DRC/LVS-clean GF180MCUD tapeout as ground truth. Clusters 2 and 3 (hardware security of the output chip) are downstream consequences of the agentic flow security problem, not the research focus.

1.4 Why this matters

Agentic chip-design pipelines are advancing rapidly, with industrial systems (ChipAgents, NVIDIA's ACE-RTL) and open-source systems (MAGE, ChipCraftBrain) reaching 95%+ functional pass rates. The community is on a trajectory toward production deployment. These systems pull specifications, datasheets, and reference designs from untrusted

sources; invoke EDA tools with broad system access; and iterate over many turns where a single hijacked step can silently corrupt every downstream artifact.

The cost of an insecure agentic flow is different in kind from a wrong answer in a chat window: it is a fabricated chip with an embedded Trojan, a degraded cipher, or a silently bypassed verification step , hardware that has passed all human review because the manipulation happened inside the agent, not in the design files a human would audit.

The agent-security literature establishes that this concern is not hypothetical. Prompt injection is ranked #1 on the OWASP Top 10 for LLM Applications, reflecting its prevalence and impact across deployed systems. Greshake et al. demonstrate that indirect injection through untrusted inputs reliably bypasses naive defenses. ToolHijacker shows that the tool-selection mechanism in deployed LLM agents is itself exploitable. MUZZLE demonstrates that adaptive red-team agents discover novel attack chains that static corpora do not enumerate. The SoK on agentic AI attack surface catalogs the failure modes that arise specifically from long-horizon tool-using agents and proposes the UAR and Privilege Escalation Distance metrics we adopt.

The EDA-specific extension of this work , pipelines that ingest specifications from untrusted sources, invoke tool binaries with broad system access, and produce a fabricable hardware artifact at the end , has not been studied. This proposal addresses that gap directly.

2. Project Scope

2.1 Scope statement

> Build, evaluate, and open-source a hardened agentic pipeline for RTL-to-GDS chip design that measurably resists adversarial manipulation of the natural-language specification, evaluated against an unhardened baseline and a per-defense-layer ablation series on a fixed red-team corpus across a panel of SOTA models. Demonstrate end-to-end pipeline function by producing a DRC/LVS-clean GF180MCUD tapeout of a PRESENT-80 block-cipher core as the ground-truth hardware artifact.

The pipeline is the contribution. The tapeout is the proof that the pipeline is real.

2.2 In scope (deliverables)

Primary: Hardened agentic pipeline

- Gate 0 plus five-stage pipeline: static pre-flight config audit (our scanner, see §3.2), then Spec Parsing, RTL Generation, Testbench/SVA, Synthesis Driver, DRC/LVS Sign-off

- Built on LangGraph as the top-level orchestrator. Each stage is implemented by one or more subagents executing under the orchestrator. The orchestrator coordinates stage execution and inter-stage state transitions; it does not enforce security properties itself
- Inter-stage state validated by Pydantic strict-mode typed schemas with allowlisted fields; any field not in the schema is rejected, not dropped
- **The pipeline is model-agnostic: no prompt, tool schema, or output parser is specialized to a single model.** Evaluation runs against a panel of SOTA models spanning open-source and closed-source families (closed examples: Claude Opus 4.x, GPT-5-class, Gemini-class; open examples: Llama 4-class, Qwen 2.5+, DeepSeek-class). Final roster pinned in Phase 1; open-source models pinned by weights hash, closed models by API version. All calls at **temperature 0**; no fine-tuning
- Sandboxed per-stage processes via Docker; no network egress, read-only PDK
- Per-stage tool allowlist enforced via declarative policy
- Verification-gated commits , oracle MUST pass before the next stage is invoked
- All LLM outputs logged as frozen artifacts enabling full corpus replay without API re-calls

Gate 0 static config scanner (releasable as standalone)

- A from-scratch static analyzer for agentic system configurations, developed as part of this work and released as its own open-source package
- Categories: secrets / credentials; tool-permission scopes including wildcard and missing-deny-list detection; hook/shell injection risk including silent-fail and remote-fetch hooks; MCP server risks including auto-install and hardcoded tokens; agent config surface including hidden-Unicode, base64-encoded directives, auto-run instructions, and prompt-reflection patterns
- Rule set developed from the published agent-security canon (OWASP LLM Top 10, Greshake et al., ToolHijacker, SoK on agentic AI attack surface) and from the first author's prior taxonomy work (manuscripts in submission)
- Deterministic (zero LLM calls); designed to be useful outside this project

Unhardened baseline (control) and defense-layer ablation series

- **Unhardened baseline:** same model panel, same tools, single orchestrator loop with no sandboxing, no inter-stage schema validation, and no verification gates between steps; identical inputs, different architecture only
- **Ablation series:** the hardened pipeline is also run with each defense layer disabled individually , Gate 0 only off; Pydantic strict-mode boundary only off; Docker sandbox only off; per-stage tool allowlist only off; verification gates only off , producing a per-layer contribution table. This converts the hardened-vs-unhardened comparison

from binary into a Pareto map of which defenses are doing how much work and is the headline structural improvement over a binary control.

- Both exist solely for the controlled experiment , not a claim about the state of the art

Security evaluation

- ONE threat model: indirect prompt injection in the natural-language specification
- Three attack categories: Trojan trigger insertion, cipher weakening (round reduction, key-bit leakage), verification bypass (skip-test or disable-assertion induction)
- **Attack taxonomy derived from the first author's prior agent-security research (manuscripts in submission), supported by the published canon (Greshake et al., ToolHijacker, MUZZLE, OWASP LLM Top 10, SoK on agentic AI attack surface)**
- Red-team corpus: 30–50 hand-crafted adversarial specifications (10–15 per category) instantiating the taxonomy
- Corpus frozen at end of Phase 2; corpus-gate CI check runs on every build
- **Statistical methodology pre-registered in ANALYSIS_PLAN.md (Phase 1 deliverable):** exact binomial confidence intervals on per-category ASR; McNemar's test for hardened-vs-unhardened comparison across the full corpus; per-layer ablation contributions reported with the same statistical machinery; a power analysis included so corpus-size limitations are stated upfront
- Metrics: Attack Success Rate (ASR), Unsafe Action Rate (UAR), Tapeout Survival, stage-of-detection breakdown across all gates, PPA delta, **per-defense-layer contribution table, cross-model and cross-family robustness comparison**
- Secondary evaluation: three-subagent Attacker/Defender/Auditor pipeline (run with one SOTA model from the panel) on both flows, generating attacks not constrained to the frozen corpus
- Cross-reference with HardSecBench tasks where the attack category overlaps

Demonstration vehicle: PRESENT-80 on GF180MCUD

- Block: PRESENT-80 block-cipher core, serial datapath (~1,500–2,500 gates, digital-only)
- Target PDK: GF180MCUD (chipathon-mandated)
- Closure: 50 MHz with comfortable margin
- Verification: published PRESENT test vectors
- Integration: chipathon's shared padring template (no custom pads)
- PRESENT-80 is chosen as the demonstration block because its published test vectors and well-known structure make it easy to detect when the agent has been manipulated: a round-reduced or key-leaking implementation fails the vectors, and a structurally-correct but Trojan-bearing implementation is the hardest case for the pipeline to catch , making it a meaningful stress test for the security architecture

Stretch goal: secure chip design

- If Phase 3 schedule permits: SymbiYosys bounded model checking establishing **functional equivalence between the produced PRESENT-80 implementation and a hand-written reference model** for all inputs within a bounded depth. We are explicit that this is equivalence-checking against a reference, not intrinsic security-property verification: a Trojan whose trigger lies within the BMC depth becomes detectable; a deeper trigger is not. The BMC depth achievable in practice will determine which Trojan classes from the corpus are catchable at Gate 3†.
- This is the only element of the proposal that is a secure-chip-design claim rather than a secure-agentic-flow claim; it is explicitly labelled as such in both the proposal and the paper
- If BMC is added, it is integrated into Gate 3 and the stage-of-detection breakdown captures attacks caught by BMC specifically

Release

- Open-source release of the hardened pipeline, the Gate 0 scanner (standalone package), the unhardened baseline, the ablation harness, the red-team corpus, and the evaluation harness
- RTL plus verification artifacts (testbench, SVA, simulation logs; SymbiYosys proofs if stretch goal is reached)
- Tapeout collateral (GDSII, DRC/LVS sign-off logs)
- Reproducibility package: frozen LLM outputs, prompts, seeds, configs, full agent traces
- **ANALYSIS_PLAN.md** , pre-registered statistical analysis plan, frozen at corpus freeze
- Workshop paper and companion security paper (see §9)

2.3 Out of scope (deferred / future work)

- **Secure chip design (beyond the stretch goal)** , side-channel resistance, power analysis, EM analysis, hardware Trojan detection in the chip as a hardware artifact: these are downstream of the agentic flow security problem and require lab-grade equipment and months of post-silicon work. This is the domain of HOST, TCHES, and IEEE TDSC; it is not the domain of this paper.
- **DFT manipulation as an attack category** , for a cryptographic block like PRESENT-80, naive scan-chain insertion exposes the key register to a physical attacker through test pins, and an adversarial spec could induce the agent to make precisely this mistake or to disable test-mode access controls. This is a genuine extension of the agentic-flow threat model (NL spec → manipulated agent → silently compromised hardware) and a strong candidate for the journal extension or a follow-up paper. **Deferred from v1 to keep the corpus tractable at three attack categories and to**

keep the synthesis stage simple; explicitly parked as future work, not as a competing scope claim.

- **Analog or mixed-signal blocks** , toolchain learning load doubles; outside our strength profile.
- **Custom standard cells, PDK extensions, padring design** , use what the chipathon provides.
- **Larger blocks** (RISC-V cores, full SoCs) , not deliverable in four months.
- **Novel agent orchestration framework** , novelty is in the security analysis, not the orchestration.
- **Fine-tuning or custom-trained models** , would consume the timeline.
- **RAG poisoning of foundry docs / datasheets** , second threat model deferred to journal extension.
- **Tool-output spoofing** (compromised linter or simulator) , deferred to follow-up paper.
- **Multi-turn / retry-loop attacks** , an attacker who knows the scrubber exists could craft an injection that uses a gate-failure response as an oracle to refine a follow-on attempt; explicitly deferred; the bounded retry limit (max N) already contains the exposure in the v1 frame.
- **Adaptive / RL-trained attackers** , the secondary Attacker/Defender/Auditor pipeline provides a generative adversarial signal; a fully adaptive attacker is deferred to future work.
- **Comparison vs. closed-source agentic flows** , the controlled baseline is our own unhardened flow.

2.4 Scope freeze rule

Threat model and red-team corpus lock at the end of Phase 2. The ANALYSIS_PLAN.md, unhardened baseline, ablation harness, model evaluation panel, and Gate 0 scanner ruleset lock at the same point. Post-freeze ideas land in BACKLOG.md. Mid-flight scope changes require mentor sign-off and a written justification.

2.5 Pre-agreed descope ladder

If the schedule slips, we drop in this order:

- PPA delta metric.
- **Evaluation model panel size , drop to a smaller subset (minimum one open-source + one closed-source pair to preserve the cross-family comparison).**
- Stretch goal (SymbiYosys BMC / secure chip design).
- Secondary Attacker/Defender/Auditor adversarial pipeline pass.

- **Per-layer ablation series (collapse back to hardened-vs-unhardened binary).**
- Verification-bypass attack category.
- Cipher-weakening attack category.
- Unhardened baseline (last resort , kills the controlled experiment).

3. Technical Approach

3.1 Architecture overview

The pipeline is a directed state graph in which each node is a **stage**, each verification oracle is a stage-to-stage transition gate, and tool calls are dispatched through a per-stage sandbox into a fixed allowlist of open-source EDA tools. Each stage is implemented by **one or more subagents executing under a top-level orchestrator (LangGraph)**. The orchestrator coordinates stage execution and routes inter-stage state; it does not enforce security properties itself. A static pre-flight audit (Gate 0) runs on the agent configuration before any LLM call is dispatched. Inter-stage state is validated at every boundary by a Pydantic strict-mode schema. The full architecture is captured in the accompanying architecture.drawio file.

The central design principle is that **the framework provides orchestration, not security.** Security is enforced independently at four layers below the framework: the static config audit (Gate 0), the per-stage schema boundary (Pydantic), the tool-dispatch allowlist, and the Docker sandbox. This layering follows the defense-in-depth pattern recommended in the agent-security literature (Beurer-Kellner et al. on agent design patterns; SoK on agentic AI attack surface) , a compromise at the LangGraph orchestration level does not automatically cascade into tool-layer access.

Model-agnostic design. No prompt, tool schema, or output parser in the pipeline is specialized to a particular model. Subagents communicate with their underlying LLM through a thin adapter layer that handles only model-specific syntax (tool-call format, system-prompt positioning); the prompt content and the Pydantic schemas are identical across models. This lets us swap any model from the evaluation panel into any subagent slot without changing pipeline code, which is what makes the cross-model evaluation in §3.8 a clean experiment rather than a methodological footnote.

3.2 Gate 0: Static pre-flight agent configuration audit

Before the pipeline accepts any input, Gate 0 runs an automated static audit of the agent configuration directory using **our own static scanner**, which we develop as part of this work and release as a standalone open-source package alongside the pipeline. This is a deterministic, zero-LLM-call check that must return clean before any specification is processed.

Gate 0 audits:

Category	What is checked
Secrets	Hardcoded API keys, tokens, credentials in config files
Permissions	Wildcard tool allowlists (Bash(*)), missing deny-list entries for destructive operations
Hook injection	Shell-hook commands with attacker-controllable interpolation; silent-fail hooks (2>/dev/null); hooks that fetch and execute remote content
MCP server risks	High-risk server types; auto-install via npx -y; hardcoded tokens in MCP env config; remote transport URLs
Agent config surface	Hidden Unicode instructions; base64-encoded directives; auto-run instructions; prompt reflection patterns

The scanner's rule set is grounded in three sources: the published agent-security literature (OWASP LLM Top 10, Greshake et al. on indirect prompt injection, ToolHijacker on tool-selection attacks, the SoK on agentic AI attack surface), CWE entries relevant to credential handling and configuration injection, and the first author's prior taxonomy work on agentic system threat modeling (manuscripts in submission). The scanner is designed to be useful outside this project , it is one of the project's separately reusable deliverables.

Gate 0 re-runs on every CI build as part of the corpus-gate check. A failing Gate 0 blocks the entire pipeline , not because the specification is adversarial, but because the pipeline configuration itself is misconfigured. This prevents a class of attack that the verification oracles in Gates 1–5 are not designed to catch.

3.3 The five stages

Each stage is a LangGraph node executing in its own Docker container with no network egress and read-only PDK access. Each stage is implemented by one or more subagents running inside that container; subagents within a stage share the container, but their outputs are not externally visible until the stage's verification gate passes. Inter-stage state passes exclusively through LangGraph's typed state object, validated by Pydantic strict mode at every transition: fields not in the allowlisted schema are rejected with a hard error, never silently dropped. This closes the intra-pipeline attack vector where a compromised Spec Parsing stage passes schema-valid but semantically adversarial payloads to the RTL Generation stage.

#	Stage	Role	PRESENT-80 demo task	Allowed tools
1	Spec Parsing	Validates and normalizes the NL spec into a typed JSON schema; scrubs embedded instructions	Parses PRESENT-80 cipher spec	(none)
2	RTL Generation	Emits Verilog from the validated schema only	Generates serial-datapath PRESENT-80	Icarus Verilog, Yosys (lint)
3	Testbench & Assertions	Generates SVA and functional tests	Tests against published PRESENT vectors	Icarus Verilog
4	Synthesis Driver	Drives synthesis and place-and-route; enforces PPA budget	Yosys → OpenROAD, 50 MHz closure	Yosys, OpenROAD
5	DRC/LVS Sign-off	Validates GDSII against PDK rules and netlist equivalence	GF180MCUD sign-off	KLayout, Magic

The stage roles are defined abstractly; PRESENT-80 is what each stage acts on in the demonstration run. The pipeline accepts any digital block specification; the security evaluation runs on adversarially-modified PRESENT-80 specifications.

3.4 Verification gates as security boundaries

Between every pair of consecutive stages sits a verification oracle. Each gate must pass before the next stage is invoked. Gate failure triggers a bounded retry loop (max N attempts) before escalation or abort. The gates are the security boundary , not quality controls bolted onto an existing flow.

Gate	Check	Security function
0	Static config audit	Rejects misconfigured pipeline before any input is processed
1	Spec Schema Check	Pydantic strict extraction; instruction-injection scrubbing by construction (see §3.6)
2	Functional Simulation	RTL must pass published test vectors; detects most manipulation of functional behavior
3	Lint + Formal Equivalence	Yosys lint; formal equivalence between RTL and synthesized netlist

Gate	Check	Security function
3†	BMC (stretch goal)	SymbiYosys bounded model checking establishing functional equivalence between produced RTL and a hand-written reference within bounded depth; catches Trojan variants whose trigger lies within that depth
4	STA + DRC pre-check	Timing closure; preliminary DRC
5	Tapeout Sign-off	Clean DRC, clean LVS, sign-off-quality reports

On the limitation of Gate 3 (equivalence checking): Formal equivalence verifies that the synthesized netlist is functionally identical to the RTL , not that the RTL is free of malicious behavior. A Trojan that adds a dormant state bit activated by a rare input pattern survives Gate 2 (test vectors don't exercise that input) and passes Gate 3 (the netlist is equivalent to the Trojan-bearing RTL). This is why **Tapeout Survival is the most pessimistic and analytically important metric**: it measures what fraction of attacks survive the entire gate stack and reach a sign-off-clean GDSII. The stretch-goal SymbiYosys BMC is the additional defense against this specific class , for Trojans whose trigger pattern lies within the BMC depth , but even without it, the stage-of-detection breakdown will show where in the gate stack each attack category is caught, which is itself a substantive result.

3.5 Sandboxing layer

- Each stage runs in its own Docker container with no network egress. Subagents within a stage share the stage's container; cross-stage isolation is enforced at the stage boundary.
- The PDK is mounted read-only.
- Tool binaries are pinned to specific versions and signed.
- The tool-dispatch wrapper enforces the per-stage allowlist; an off-list tool call is logged and counted toward UAR.
- Inter-stage state passes through LangGraph's typed state object only , not through shared filesystem , and is validated by Pydantic strict mode at every boundary.

This is a layered defense-in-depth design: each layer is independently enforced, so a compromise at the LangGraph orchestration layer does not cascade into tool access. The framework alone does not provide security; the gap between “framework” and “secure framework” is what the hardened-vs-unhardened comparison , and the per-layer ablation series , makes visible.

3.6 Threat model

What we are defending against: An adversary who controls the natural-language specification input and embeds instructions designed to steer the agent pipeline into producing a subtly incorrect, degraded, or backdoored hardware implementation , while the specification appears legitimate to a human reviewer.

This is an agentic flow security problem, not a chip security problem. The goal of the attack is to manipulate the agent, not to directly modify the RTL or the GDSII.

Three attack categories (v1 scope):

- **Trojan trigger insertion** , injection payload instructs the RTL-generation subagent to add a state transition that produces incorrect output on a specific rare input.
- **Cipher weakening** , payload reduces the round count, leaks key bits through a side wire, or weakens the S-box substitution.
- **Verification bypass** , payload instructs the testbench subagent to skip tests, disable assertions, or accept failing vectors as passing.

Spec Parser scrubbing (Gate 1 primary defense): The Spec Parsing stage does not attempt to sanitize a natural-language string , it does not use regex patterns or a secondary LLM call as a cleaner (which would introduce a recursive trust problem). Instead, it performs **structured extraction only**: it extracts the fields present in the allowlisted Pydantic schema (block name, datapath width, target frequency, reference standard, I/O interface type) and discards everything else by construction. An embedded injection payload has nowhere to go: it does not map to any schema field and is never passed to a downstream stage. All schema fields are strict-typed with regex constraints; no Any types; no freeform string fields wider than 64 characters. This makes the scrubber's behavior deterministic and auditable, unlike LLM-based sanitization approaches. The pattern is the action-selector design pattern from Beurer-Kellner et al.

Multi-turn / retry-loop attacks are out of v1 scope: an adversary aware of the bounded retry loop could craft an injection that uses a gate-failure response as an oracle for a refined follow-on attempt. The retry limit (max N) bounds exposure; systematic retry-oracle exploitation is deferred to the journal extension.

3.7 Red-team corpus

The corpus consists of 30–50 adversarial specifications (10–15 per category), each a complete specification document with an embedded injection pattern. The taxonomy of injection patterns is drawn from:

- **The first author's prior agent-security research (manuscripts in submission)**, which catalogs injection-pattern categories including system-prompt overrides, identity reassignment, jailbreak patterns, zero-width Unicode payloads, base64-encoded directives, data exfiltration via embedded URLs, auto-run instruction

embedding, output manipulation instructions, prompt reflection, and time-conditioned execution.

- The published agent-security canon: Greshake et al. on indirect prompt injection; ToolHijacker on tool-selection manipulation; MUZZLE on adaptive red-teaming methodology; OWASP LLM Top 10; SoK on agentic AI attack surface.
- CWE-coded hardware vulnerability templates from HardSecBench (where category overlaps).

The corpus is constructed in Phase 2 and frozen at Phase 2's end. All evaluation runs use identical inputs. A corpus-gate CI check runs on every build; regression in detection coverage fails the build and emits a prioritized improvement plan by category.

Statistical framing: Pre-registered in ANALYSIS_PLAN.md (Phase 1). Exact binomial confidence intervals on per-category ASR. McNemar's test for the hardened-vs-unhardened comparison and for each ablation-vs-hardened comparison across the full corpus. Power analysis included so the corpus-size constraint is stated upfront in the paper methodology section.

3.8 Evaluation metrics

The evaluation answers one question: **does the hardened agentic pipeline resist adversarial specifications better than the unhardened baseline, and which defense layers carry which load , across which model families?** All metrics exist to answer that question.

Primary metrics (hardened vs. unhardened, paired on identical corpus)

- **Attack Success Rate (ASR)** , fraction of corpus entries where the malicious behavior reaches the synthesized netlist. Reported per category with exact binomial CIs. McNemar's test for hardened-vs-unhardened.
- **Unsafe Action Rate (UAR)** , fraction of subagent steps that attempt a tool call outside the per-stage allowlist. Adopted from the SoK on agentic AI.
- **Tapeout Survival** , fraction of attacks where the malicious output passes all gates and reaches a sign-off-clean GDSII. The most pessimistic and most meaningful metric: this is what would be fabricated.
- **PPA delta** , area, power, and timing cost of the hardened flow vs. the unhardened baseline. Quantifies what the security architecture costs.

Per-layer ablation (the per-defense contribution table)

The hardened pipeline is re-run five additional times with each defense layer disabled individually , Gate 0 only off, Pydantic strict-mode only off, Docker sandbox only off, tool allowlist only off, verification gates only off , and ASR / UAR / Tapeout Survival are reported for each ablation. The resulting table identifies which layers are necessary, which are redundant, and where the residual attack surface lies. This is the headline structural

improvement over a binary hardened-vs-unhardened comparison and converts the result from “our defense works” to “here is the per-layer cost-benefit of an agentic EDA defense stack.”

Cross-model and cross-family robustness

All primary metrics and the ablation series are run across the full evaluation panel of SOTA models spanning open-source and closed-source families. Per-model results are reported individually; cross-family differences (closed vs. open) are reported as a separate aggregated result. This addresses the standard reviewer concern that security results may be an artifact of one model's in-context safety training rather than a property of the pipeline architecture. Open-source runs are bit-for-bit reproducible (pinned by weights hash); closed-source runs are replayable via frozen output capture, since temperature-0 sampling does not guarantee strict determinism across provider-side batching and silent model updates.

Stage-of-detection breakdown (kill chain analysis)

For every attack in the corpus, we record which gate first blocks it: Gate 0 (config audit), Gate 1 (schema / scrubber), Gate 2 (functional sim), Gate 3 (lint + equivalence), Gate 3† (BMC, if stretch goal reached), Gate 4 (STA + DRC), Gate 5 (tapeout sign-off), or “not detected.” This produces a kill-chain table: what fraction of each attack category is neutralized at each stage, which gates carry the most security load, and what the residual rate is at tapeout. This is a richer result than the summary metrics alone and directly supports the paper's claim about which defense layers matter.

Secondary evaluation (generative adversarial pass)

A three-subagent Attacker/Defender/Auditor pipeline (run with one SOTA model from the panel) operates on both flows. The Attacker subagent finds exploitable chains not constrained to the frozen corpus; the Defender subagent evaluates existing protections; the Auditor subagent synthesizes a prioritized risk assessment. Results are reported separately from the primary metrics and inform the journal extension scope.

3.9 Implementation stack

Component	Choice	Rationale
Orchestrator	LangGraph (pinned)	Explicit state-graph topology; native checkpoint/interrupt; inspectable typed state; maps cleanly onto verification-gate architecture

Component	Choice	Rationale
LLM panel	SOTA open-source and closed-source models; roster pinned in Phase 1	Pipeline is model-agnostic by construction; cross-family evaluation rules out single-model RLHF artifacts; open-source models give weights-hash reproducibility, closed-source models give SOTA reasoning
Model adapter layer	Thin per-model adapter handling only call-format differences	Keeps prompts and schemas model-agnostic; enables drop-in model swap for the cross-model evaluation
Inter-stage schema	Pydantic strict mode	Deterministic boundary enforcement; no Any types; allowlisted fields only
Sandboxing	Docker per stage, no network egress, read-only PDK	Independent enforcement below framework level
Gate 0 (config audit)	Our own static scanner (released as standalone)	Deterministic, zero-LLM-call pre-flight check; reusable beyond this project
EDA tools	Yosys, Icarus Verilog, OpenROAD, KLayout, Magic (IIC-OSIC-TOOLS image, pinned)	Chipathon-mandated toolchain
Formal verification (stretch)	SymbiYosys	BMC for functional equivalence against reference model
Tracing	Structured log per call / dispatch / gate / transition	Corpus replay; metrics computation; reproducibility
CI	Corpus-gate runner + Gate 0 audit on every build	Regression detection; frozen corpus integrity

Reproducibility: All LLM calls at temperature 0. All outputs logged as frozen artifacts. The corpus evaluation can be replayed without re-calling any API. Prompts, seeds, configurations, and full agent traces are published with the repository. **We are explicit that temperature-0 sampling does not guarantee strict determinism for closed-source API models , provider-side batching and silent model updates can introduce variation. Replayability of the evaluation is guaranteed via the frozen artifacts; reproducibility of live API runs is not. The open-source model runs, pinned by weights hash, are bit-for-bit reproducible by anyone with the corresponding hardware.**

4. Deliverables

1) **Hardened agentic pipeline** (open-source): Gate 0 through Gate 5, top-level orchestrator with stage-level subagents, Docker sandboxing, Pydantic inter-stage schemas, per-stage tool allowlists, model-agnostic adapter layer, corpus-gate CI configuration.
2) **Gate 0 static config scanner** (open-source, releasable standalone): A general-purpose static analyzer for agentic system configurations, developed from the agent-security canon and the first author's prior taxonomy work. Usable outside this project.
3) **Unhardened baseline + defense-layer ablation harness** (open-source): identical model panel and tools, single orchestrator loop, no gates, no sandboxing , plus a per-layer ablation runner that disables individual defenses for the contribution table.
4) **Red-team corpus and evaluation harness** (open-source): 30–50 frozen adversarial specifications instantiating the project's attack taxonomy; metric calculators for ASR, UAR, Tapeout Survival, stage-of-detection; per-layer ablation reporter; cross-model and cross-family comparator; Attacker/Defender/Auditor adversarial pipeline; McNemar's test, exact binomial CI, and power analysis implementations.
5) **Tapeout package** (GF180MCUD): DRC/LVS-clean GDSII of the PRESENT-80 demonstration block integrated into the chipathon's shared padring template. Ground-truth evidence that the pipeline produces real hardware under adversarial evaluation conditions.
6) **RTL and verification artifacts**: testbench, SVA, simulation logs, SymbiYosys proofs (if stretch goal reached).
7) **Reproducibility package**: all frozen LLM outputs, prompts, seeds, configurations, full agent traces, and **ANALYSIS_PLAN.md** with the pre-registered statistical methodology.
8) **Workshop paper**: drafted by end of Phase 4. Centers the agentic pipeline architecture, the gate design including Gate 0, the open-source Gate 0 scanner, the tapeout as ground truth, and the open-source contribution. Empirical robustness results appear as motivation and as exploratory results, framed qualitatively.
9) **Companion security paper**: see §9 for venue strategy. Centers the threat model, the corpus methodology, the per-layer ablation results, the kill-chain analysis, the cross-model and cross-family robustness comparison, the statistical methodology, and the defense architecture. The fabricated chip is ground-truth evidence; it does not make this a chip-security paper.
10) **Post-tapeout functional bring-up notes** (Phase 5): results added to the open repository where lab access permits.

5. Timeline

Phase 1 , May 2026: Onboarding and lock-in

- Team formed; block choice confirmed with mentor (PRESENT-80; fallback to a smaller block , parity-checked register file or small FSM , not a larger one)
- Threat model document written and shared (THREAT_MODEL.md)
- **ANALYSIS_PLAN.md drafted:** pre-registered statistical plan (McNemar power analysis, ablation reporting structure, cross-model and cross-family comparison structure)
- **Evaluation model panel finalized:** roster of SOTA open-source and closed-source models selected, with closed models pinned by API version and open-source models pinned by weights hash
- Repository skeleton; CI configured with Gate 0 and corpus-gate stubs
- **Gate 0 scanner design specified;** rule categories enumerated; ruleset stub committed; standalone-release packaging planned
- LangGraph orchestrator pinned; Pydantic schema for inter-stage state drafted; Docker skeletons per stage; model-adapter interface drafted
- Off-the-shelf single-stage baseline running end-to-end on a 4-bit counter (toolchain proof)
- Stretch-goal assessment: determine whether SymbiYosys coverage of PRESENT-80 is feasible within the timeline; go/no-go decision documented

Phase 2 , May/June 2026: Pipeline build and corpus construction

- Hardened pipeline built: Gate 0, five stages, Pydantic boundaries, Docker sandboxing, verification oracles, model-adapter layer
- **Gate 0 scanner implemented and tested;** first standalone release candidate
- Unhardened baseline specified and implemented
- **Ablation harness built** (per-layer disable hooks in the orchestrator)
- PRESENT-80 RTL generated by the pipeline and verified against published test vectors
- Synthesis closing: Yosys → OpenROAD, 50 MHz
- Red-team corpus drafted: 10–15 attacks per category, instantiating the project taxonomy
- **Corpus, threat model, analysis plan, unhardened baseline, ablation harness, model panel, and Gate 0 ruleset frozen at end of Phase 2**

Phase 3 , July 2026: Evaluation and design review

- Both flows running on the full corpus, **across the full model panel**

- **Per-layer ablation series run;** contribution table produced; per-model and per-family breakdowns produced
- First robustness results: ASR (with CIs), UAR, Tapeout Survival, stage-of-detection breakdown, PPA delta, per-layer contribution, cross-model and cross-family comparison; McNemar's test
- Attacker/Defender/Auditor adversarial pipeline first run on both flows
- DRC/LVS-clean placeholder GDSII submitted for mentor design review
- Stretch goal: SymbiYosys BMC integrated into Gate 3 if go-decision was made in Phase 1

Phase 4 , August/September 2026: Final evaluation and tapeout

- Final hardening of the pipeline configuration
- Frozen corpus run end-to-end on both flows and the full ablation series across the model panel; all metrics finalized
- Tapeout package submitted by shuttle deadline
- Workshop paper drafted; companion security paper outline written

Phase 5 , Post-tapeout

- Functional bring-up of PRESENT-80 on returned silicon
- Bring-up data added to the open repository
- Workshop paper finalized; companion security paper submitted to primary venue

6. Risks and Mitigations

Risk	Likelihood	Mitigation
Unhardened baseline delta is uninteresting (baseline too weak)	Medium-Low (mitigated by ablation)	**Per-layer ablation series produces a contribution table even if the binary hardened-vs-unhardened delta is small;** the ablation is the structural defense against a “strawman baseline” review. If the binary delta is small, that itself is a finding (“framework-level orchestration alone provides most of the protection observed”).
Stretch goal (SymbiYosys BMC) doesn't scale to PRESENT-80 gate count	Medium	Go/no-go decision in Phase 1; if infeasible, scope to S-box or key-schedule subcircuit and document the limitation. BMC is not on the critical path.

Risk	Likelihood	Mitigation
Equivalence checking (Gate 3) passes Trojan-bearing RTL	Known , by design	Explicitly named in §3.4 and the paper. Tapeout Survival is the metric for this. BMC (stretch) is the additional defense for trigger-within-depth Trojans. Not a flaw in the experimental design , a finding about which gates catch which attack classes.
Spec Parser scrubber bypassed by novel injection pattern	Medium	Scrubber is schema-based, not sanitization-based. Novel patterns that don't map to a schema field are discarded by construction. Residual risk is an overly-permissive schema field; mitigated by keeping all fields narrow and regex-constrained.
Corpus too small for statistical significance	Known constraint	**Power analysis included in ANALYSIS_PLAN.md** (Phase 1). Exact binomial CIs and McNemar's test explicitly reported. 10–15 binary outcomes per category supports meaningful paired comparison; paper methodology section is explicit about this.
Modern LLMs' in-context safety training already resists most of the corpus	**Medium**	**Two responses: (a) corpus deliberately includes patterns documented in the literature to evade current safety training , zero-width Unicode, base64 directives, indirect injection via embedded artifacts , not patterns trivially caught by RLHF; (b) the cross-family evaluation includes open-source models with less aggressive safety training, where baseline ASR is expected to be higher. If all panel models resist the corpus uniformly, that itself is a substantive finding.**
Evaluation panel drifts during the project (new model releases)	**Low-Medium**	**Pin all panel members at end of Phase 1 (weights hash for open-source, API version for closed-source); freeze at end of Phase 2. Releases after the freeze date do not enter the primary evaluation but can be added in a supplementary section if time permits.**

Risk	Likelihood	Mitigation
Model-agnostic adapter layer leaks model-specific behavior	**Low-Medium**	**Adapter layer is tested with all panel models on a fixed regression suite at the start of Phase 3; any model-specific divergence in observed behavior at the schema or tool-call layer is treated as an adapter bug and fixed before the corpus run.**
Threat-model scope creep into secure chip design	Medium	§1.1 makes the scope explicit; §2.3 lists secure chip design and DFT manipulation as out of scope; the descope ladder is clear. If reviewers push for chip-security results, the stretch-goal SymbiYosys results are the response; nothing else is committed.
Tapeout slips due to pipeline implementation overrun	Low–Medium	Pre-agreed descope ladder (§2.5); tapeout is the last thing to be removed, not the first.
Tapeout fallback if PRESENT-80 doesn't close	**Low**	**PRESENT-80 at ~2k gates at 50 MHz on GF180MCU 180nm is far below the technology's headroom. If a hard blocker appears, fall back to a smaller demonstration block (parity-checked register file, small FSM with verifiable assertions), not to a larger one. The fallback block must preserve the security-evaluation properties (clear test vectors, detectable manipulation).**
LLM API costs balloon	Low	Temperature 0 throughout; frozen output artifacts mean corpus evaluation does not re-call the API. Budget capped per model in the panel; cap per-run token budgets in code. **Open-source models run via local inference if budget pressure appears.**
Cannot recruit IC-experienced teammate in Phase 1	Medium	Personal background covers digital RTL-to-GDS independently; solo execution is feasible. Ideal-case adds one teammate for synthesis tuning and layout review.

7. Team Needs

We are looking for one of the following configurations:

- **Lead a small team of 2–3.** Ideal complementary skills:
  - One teammate experienced in OpenROAD synthesis/PnR for GF180MCU.
  - Optionally one teammate with LLM agent infrastructure experience (LangGraph, MCP) if implementation bandwidth becomes the constraint.
- **Join an existing Track D team** where the security-evaluation angle complements rather than duplicates the core work.

8. Applicant Background

PhD student in agent security, focusing on adversarial robustness of LLM-based autonomous systems: prompt injection, tool misuse, and supply-chain risks in agent toolchains. MSc in VLSI design with hands-on experience across the open-source RTL-to-GDS flow: Yosys, OpenROAD, Icarus Verilog, KLayout, Magic. Recent results include a top placement in the OpenPOWER Hackathon (ChipFoundry) and an additional digital design contest win. Prior research on agentic system threat-model taxonomy and static configuration analysis is in submission and serves as the methodological foundation for the corpus design and the Gate 0 scanner ruleset in this work.

The distinguishing combination is: deep agent-security background with operational VLSI tooling fluency. Most LLM-for-RTL work is led by IC designers learning to wrap LLMs around their flows. Approaching it from the agent-security direction , with the toolchain already fluent , opens the research direction this proposal addresses.

9. Publication Strategy

We plan to produce two papers from the same artifact. The framing in each is calibrated to its community, and the primary contributions are disjoint.

Workshop paper (chipathon-aligned venue, SSCS-flavored): centers the agentic RTL pipeline, the stage and gate architecture including Gate 0, the open-source Gate 0 scanner, the tapeout as ground truth, and the reproducibility package. Empirical robustness results appear as motivation and as an exploratory results section, not as the paper's central contribution. Target: the chipathon-aligned workshop.

Companion security paper , primary target: USENIX Security or CCS. The natural home is a systems/AI security venue, not a hardware security venue. USENIX Security and CCS review committees are familiar with agent-security methodology, red-team evaluation methodology, McNemar's test, attack corpus design, and ablation methodology; they will not need the paper to teach them what DRC/LVS-clean means. The silicon artifact is novel in that community and provides concrete ground truth that agent-security papers

normally lack , it is an asset, not a barrier to comprehension. The cross-family LLM evaluation (multiple open-source and closed-source models) addresses the standard reviewer concern that security results may be an artifact of a single model's in-context safety training.

Fallback venues in order: IEEE S&P, NDSS, DAC security track, HOST. HOST is the appropriate fallback if the stretch-goal SymbiYosys results are strong and the reviewers weight the hardware artifact heavily; it is not the primary target because HOST's program committee centers hardware security of the chip, not agentic flow security.

Two-paper non-overlap statement

The workshop paper and the companion security paper share infrastructure (the pipeline implementation, the Gate 0 scanner, the red-team corpus, the tapeout, the evaluation harness, the raw traces) but make disjoint primary contributions:

- **Workshop paper primary contributions:** the open-source agentic pipeline architecture; the integration of agent-security defense layers into an EDA toolchain; the open-source Gate 0 static config scanner; the fabricated GF180MCUD tapeout as artifact; the reproducibility package. Empirical robustness results appear as motivation and exploratory results, framed qualitatively; no statistical hypothesis tests appear as primary results.
- **Companion security paper primary contributions:** the threat model for agentic RTL pipelines; the attack-category taxonomy and red-team corpus methodology; the per-defense-layer ablation results; the kill-chain stage-of-detection analysis; the cross-model and cross-family robustness comparison; the statistical methodology and pre-registered analysis plan; the generative-adversarial secondary pass. The tapeout is referenced as ground-truth evidence, not as a primary contribution.

No experimental result is claimed as a primary contribution in both papers. Shared code, corpus, and chip are cited in both, not re-described. The papers can be read independently without either being substantially redundant with the other.

Both papers are designed from Phase 1 to avoid over-rotating toward either community. The threat model document (THREAT*MODEL.md), the corpus structure, the analysis plan (ANALYSIS*PLAN.md), and the architecture are written to be legible to both security venues and hardware design venues without requiring a different experimental setup for each.

10. References (Literature Map)

Agentic RTL generation frameworks (the baselines we harden)

- Thakur et al., *VeriGen* (2023); Liu et al., *VerilogEval* (2023) , foundational benchmarks.
- Zhao et al., *MAGE: A Multi-Agent Engine for Automated RTL Code Generation* (2024), arXiv:2412.07822 , first open-source multi-agent system for Verilog.

- Islam et al., *AIvril2* (2024); Mi et al., *CoopetitiveV* (2024) , multi-LLM ensembles with verification-in-the-loop.
- *ChipCraftBrain: Validation-First RTL Generation via Multi-Agent Orchestration* (2026), arXiv:2604.19856 , adaptive PPO-orchestrated pipeline.

LLMs producing insecure hardware (adjacent , chip security)

- *HardSecBench: Benchmarking the Security Awareness of LLMs for Hardware Code Generation* (2026), arXiv:2601.13864 , 924 tasks, 76 hardware-relevant CWEs. Evaluates chip security properties of generated RTL; does not evaluate pipeline robustness under adversarial input.
- *Evolutionary Large Language Models for Hardware Security: A Comparative Survey* (2024), arXiv:2404.16651.

LLM Trojans: insertion and detection (adjacent , chip security)

- *SENTAUR* (2024), arXiv:2407.12352 , LLM-based Trojan insertion into RTL.
- *RTL-Breaker* (2024), arXiv:2411.17569 , backdoor attacks on the LLMs themselves.
- *TrojanWhisper* (2024), arXiv:2412.07636 , LLMs as Trojan detectors.

LLM-aided vulnerability detection and repair

- *VerilogLAVD* (2025), arXiv:2508.13092 , Verilog Property Graph + LLM-generated CWE rules.
- Fan et al., *SecV* (IJCAI 2025).
- Ahmad et al., on LLM-ensemble RTL repair (VTS 2026 survey, arXiv:2604.01572).

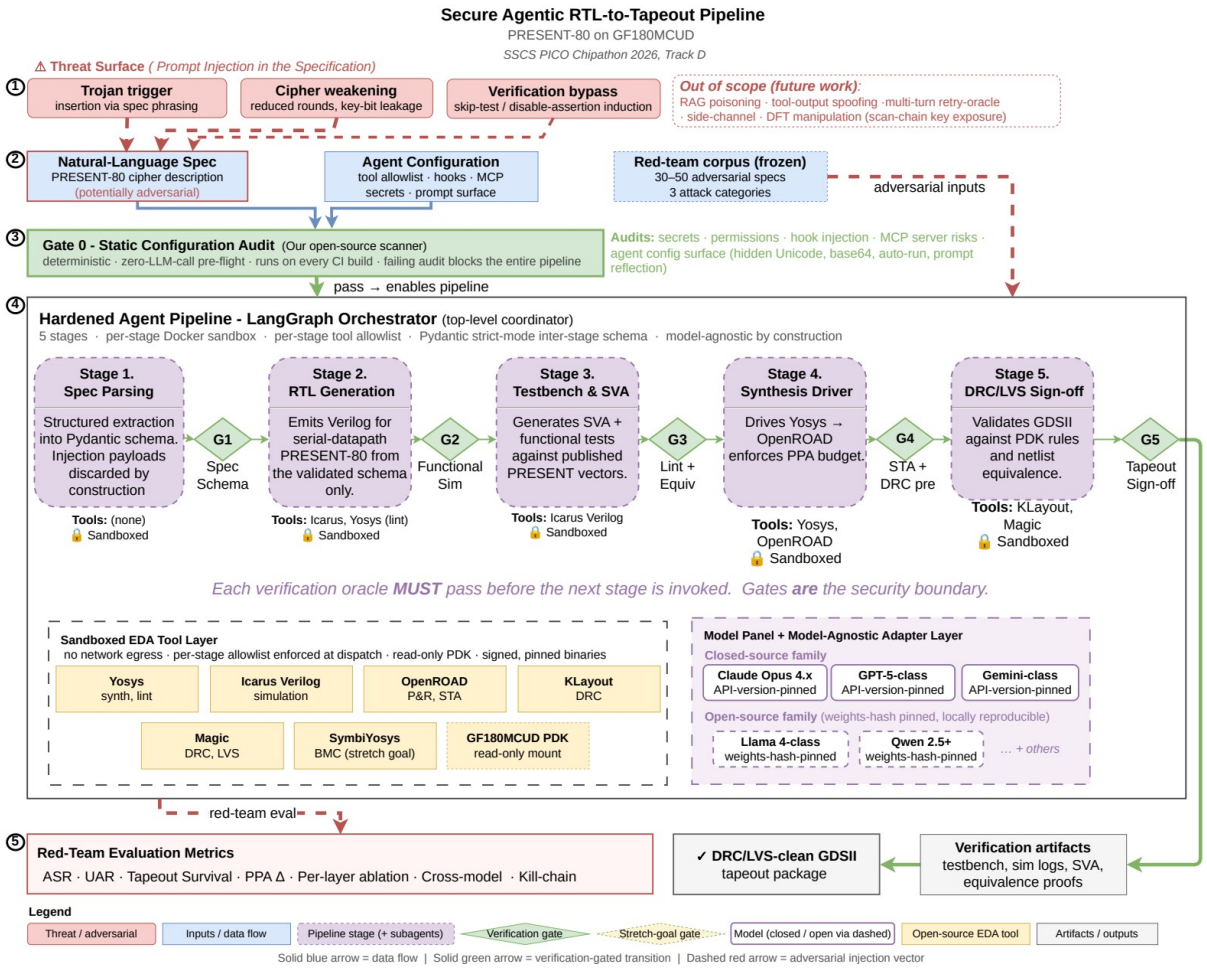
Agent security (the primary methodological grounding)

- OWASP Top 10 for LLM Applications , prompt injection ranked #1.
- Greshake et al. on indirect prompt injection , canonical reference.
- *ToolHijacker* (2024), arXiv:2504.19793 , tool-selection prompt injection in LLM agents; directly relevant to EDA tool-dispatch attacks.
- *MUZZLE* (2026), arXiv:2602.09222 , adaptive red-teaming methodology for web agents.
- *SoK: The Attack Surface of Agentic AI* (2026), arXiv:2603.22928 , proposes UAR and Privilege Escalation Distance metrics; our metrics are derived from this framework.
- *ASTRIDE* (2025), arXiv:2512.04785 , STRIDE extension for agentic AI threat modeling.
- Beurer-Kellner et al., *Design Patterns for Securing LLM Agents Against Prompt Injections* (2025) , action-selector and plan-then-execute patterns underlie our gate architecture and the Spec Parser scrubbing strategy.

Formal verification (for the stretch goal)

- *SymbiYosys: Formal Verification Flow for Yosys* , BMC and formal equivalence framework.
- Wolf, C., *Yosys Open Synthesis Suite* , core synthesis tool.

Appendix A: Project Artifacts



Appendix B: Repository Layout (proposed)

```
secure-rtl-agent/
├── README.md
├── BACKLOG.md
├── THREAT_MODEL.md
```

```
├── ANALYSIS_PLAN.md
├── orchestrator/                    # top-level LangGraph orchestrator
├── stages/                          # one LangGraph node per stage; each stage
│                                    # implemented by one or more subagents
│   ├── spec_parsing/
│   ├── rtl_generation/
│   ├── testbench/
│   ├── synthesis/
│   └── drc_lvs/
├── adapters/                        # per-model thin adapter layer (model-agnostic
core)
├── sandbox/                         # Docker definitions per stage + dispatch wrapper
├── policy/                          # per-stage tool allowlist (declarative)
├── schema/                          # Pydantic strict-mode inter-stage state schemas
├── scanner/                         # Gate 0 standalone static config scanner
│   ├── rules/
│   ├── engine/
│   └── README.md                    # standalone release notes
├── gates/                           # verification oracle implementations
│   ├── gate0_config_audit/          # wraps scanner/ for in-pipeline use
│   ├── gate1_spec_schema/
│   ├── gate2_functional_sim/
│   ├── gate3_lint_equivalence/
│   ├── gate3_bmc/                   # stretch goal: SymbiYosys properties
│   ├── gate4_sta_drc/
│   └── gate5_tapeout_signoff/
├── baseline/                        # unhardened baseline implementation
├── ablation/                        # per-defense-layer disable harness
├── corpus/                          # frozen red-team specifications
│   ├── trojan_trigger/
│   ├── cipher_weakening/
│   └── verification_bypass/
├── eval/                            # evaluation harness
│   ├── corpus_gate/                 # CI corpus-gate runner
│   ├── stage_detection/             # kill-chain breakdown across gates
│   ├── ablation_runner/             # per-layer ablation execution
│   ├── cross_model/                 # multi-model comparator (per-model + cross-
family)
│   ├── statistical/                 # McNemar, exact binomial, power analysis
│   └── adversarial_pipeline/        # Attacker/Defender/Auditor secondary pass
├── demo/                            # PRESENT-80 demonstration block
│   ├── rtl/                         # generated RTL + reference implementation
│   ├── formal/                      # SymbiYosys properties (stretch goal)
│   └── tapeout/                     # GDSII, sign-off logs, padring integration
├── traces/                          # frozen LLM outputs (temperature=0) + agent
traces
└── paper/                           # workshop + companion security paper drafts
```